

Syscalls y señales

Sistemas Operativos
DC - FCEN - UBA

13 de agosto de 2026

- ¿Cómo interactuamos con el SO?
- ¿Qué son las señales y cómo utilizarlas?
- Ingeniería inversa con [strace](#).

¿Cómo interactuamos con el SO?

Recordemos

- Como **usuarios**: programas o utilidades de sistema.

¹Portable Operating System Interface [for UNIX]

¿Cómo interactuamos con el SO?

Recordemos

- Como **usuarios**: programas o utilidades de sistema.
Por ejemplo: `ls`, `time`, `mv`, `who`, etc.

¹Portable Operating System Interface [for UNIX]

¿Cómo interactuamos con el SO?

Recordemos

- Como **usuarios**: programas o utilidades de sistema.
Por ejemplo: `ls`, `time`, `mv`, `who`, etc.
- Como **programadores**: llamadas al sistema o `syscalls`.

¹Portable Operating System Interface [for UNIX]

¿Cómo interactuamos con el SO?

Recordemos

- Como **usuarios**: programas o utilidades de sistema.
Por ejemplo: `ls`, `time`, `mv`, `who`, etc.
- Como **programadores**: llamadas al sistema o `syscalls`.
Por ejemplo: `time()`, `open()`, `write()`, `fork()`, `wait()`, etc.

¹Portable Operating System Interface [for UNIX]

¿Cómo interactuamos con el SO?

Recordemos

- Como **usuarios**: programas o utilidades de sistema.
Por ejemplo: `ls`, `time`, `mv`, `who`, etc.
- Como **programadores**: llamadas al sistema o `syscalls`.
Por ejemplo: `time()`, `open()`, `write()`, `fork()`, `wait()`, etc.

Ambos mecanismos suelen estar estandarizados.

Linux sigue el estándar **POSIX**¹.

¹Portable Operating System Interface [for UNIX]

- Las **syscalls** proveen una **interfaz** a los servicios brindados por el sistema operativo: la API (Application Programming Interface) del SO.
- La mayoría de los programas hacen un uso intensivo de ellas.

- Las **syscalls** proveen una **interfaz** a los servicios brindados por el sistema operativo: la API (Application Programming Interface) del SO.
- La mayoría de los programas hacen un uso intensivo de ellas.
- Implementación: en general, se usa una interrupción para pasar a modo **kernel**, y los parámetros se pasan usando registros o una tabla en memoria. En Linux: interrupción **0x80** (en 32 bits); el **número de syscall** va por EAX (o RAX).

- Las **syscalls** proveen una **interfaz** a los servicios brindados por el sistema operativo: la API (Application Programming Interface) del SO.
- La mayoría de los programas hacen un uso intensivo de ellas.
- Implementación: en general, se usa una interrupción para pasar a modo **kernel**, y los parámetros se pasan usando registros o una tabla en memoria. En Linux: interrupción **0x80** (en 32 bits); el **número de syscall** va por EAX (o RAX).
- Normalmente se las utiliza a través de **wrapper functions** en C. ¿Por qué no directamente? Veamos un ejemplo.

Un primer ejemplo en x86

tinyhello.asm

```
section .data
hello: db 'Hola S0!', 10
hello_len: equ $-hello

section .text
global _start
_start:
    mov eax, 4 ; syscall write
    mov ebx, 1 ; stdout
    mov ecx, hello ; mensaje
    mov edx, hello_len
    int 0x80

    mov eax, 1 ; syscall exit
    mov ebx, 0 ;
    int 0x80
```

Un primer ejemplo en ARM

tinyhelloARM.asm

```
.data
/* char msg[10] = "Hola S0!" */
msg dbc "Hola S0!", 0
msglen = . - msg

.text
.global main                                /* Entrypoint */

print:
    mov r7, #0x900004                       /* Call write() */
    mov r0, #1
    svc $0                                   /* invoke syscall */

quit:
    mov r7, #0x900001                       /* Call sys_exit() */
    svc $0
```

Usando *wrapper functions* en C

- Claramente, el código anterior no es **portable**.
- Además, realizar una **syscall** de esta forma requiere programar en lenguaje ensamblador.

Usando *wrapper functions* en C

- Claramente, el código anterior no es [portable](#).
- Además, realizar una [syscall](#) de esta forma requiere programar en lenguaje ensamblador.

El ejemplo anterior, pero ahora en C:

hello.c

```
#include <unistd.h>

int main(int argc, char* argv[]) {
    write(1, "Hola SO!\n", 9);
    return 0;
}
```

Usando *wrapper functions* en C

- Claramente, el código anterior no es **portable**.
- Además, realizar una **syscall** de esta forma requiere programar en lenguaje ensamblador.

El ejemplo anterior, pero ahora en C:

hello.c

```
#include <unistd.h>

int main(int argc, char* argv[]) {
    write(1, "Hola SO!\n", 9);
    return 0;
}
```

- Las **wrapper functions** permiten interactuar con el sistema con mayor **portabilidad** y **sencillez**.

- Las **syscalls** las utilizamos mediante la biblioteca estándar de POSIX: `unistd.h`

- Las **syscalls** las utilizamos mediante la biblioteca estándar de POSIX: `unistd.h`
- La biblioteca estándar de C incluye funciones que no son **syscalls**, pero las incluyen e invocan en su código. Por ejemplo, `printf()` invoca a la **syscall** `write()`.

- Las **syscalls** las utilizamos mediante la biblioteca estándar de POSIX: `unistd.h`
- La biblioteca estándar de C incluye funciones que no son **syscalls**, pero las incluyen e invocan en su código. Por ejemplo, `printf()` invoca a la **syscall** `write()`.
- Puede verse una lista de todas ellas usando `man syscalls`.

- Las **syscalls** las utilizamos mediante la biblioteca estándar de POSIX: `unistd.h`
- La biblioteca estándar de C incluye funciones que no son **syscalls**, pero las incluyen e invocan en su código. Por ejemplo, `printf()` invoca a la **syscall** `write()`.
- Puede verse una lista de todas ellas usando **man syscalls**.

¿Qué es un programa?

Un conjunto de instrucciones diseñadas para realizar una tarea, almacenadas en la memoria.

¿Qué es un programa?

Un conjunto de instrucciones diseñadas para realizar una tarea, almacenadas en la memoria.

¿Qué es un proceso?

Un proceso es una instancia de un programa que está en ejecución, incluyendo su estado y recursos asignados.

¿Qué es un programa?

Un conjunto de instrucciones diseñadas para realizar una tarea, almacenadas en la memoria.

¿Qué es un proceso?

Un proceso es una instancia de un programa que está en ejecución, incluyendo su estado y recursos asignados.

Múltiples procesos

Varios procesos pueden ejecutar el mismo programa simultáneamente.

Process ID

- Cada proceso se le otorga un único identificador, el número **PID** (*Process ID*).
- Podemos preguntar al SO qué número de PID tiene el proceso en ejecución usando la *syscall* `getpid()`.

pid.c

```
#include <unistd.h>

int main(int argc, char* argv[]) {
    pid_t pid = getpid(); // pid_t es un renombre de int
    printf("Mi PID es %d\n", pid);
    return 0;
}
```

- `pid_t fork(void)`: Crea un nuevo proceso, que es un clon del primero.

Código en C con fork

Crea un nuevo proceso que es un clon del primero.

Proceso A

```
int main()
{
    ➡ printf("Hola SO!\n");
    fork();
    printf("Nos vemos!\n");
    return 0;
}
```

Código en C con fork

Crea un nuevo proceso que es un clon del primero.

Proceso A

```
int main()
{
    printf("Hola SO!\n");
    ➡ fork();
    printf("Nos vemos!\n");
    return 0;
}
```

Código en C con fork

Crea un nuevo proceso que es un clon del primero.

Proceso A

```
int main()
{
    printf("Hola SO!\n");
    ➡ fork();
    printf("Nos vemos!\n");
    return 0;
}
```

Proceso B

```
int main()
{
    printf("Hola SO!\n");
    ➡ fork();
    printf("Nos vemos!\n");
    return 0;
}
```

Código en C con fork

Crea un nuevo proceso que es un clon del primero.

Proceso A

```
int main()
{
    printf("Hola SO!\n");
    ➡ fork();
    printf("Nos vemos!\n");
    return 0;
}
```

Proceso B

```
int main()
{
    printf("Hola SO!\n");
    ➡ fork();
    printf("Nos vemos!\n");
    return 0;
}
```

- El proceso A se denomina Padre y el proceso B Hijo. Cada uno tiene su propio PID.

Código en C con fork

Crea un nuevo proceso que es un clon del primero.

Proceso A

```
int main()
{
    printf("Hola SO!\n");
    ➡ fork();
    printf("Nos vemos!\n");
    return 0;
}
```

Proceso B

```
int main()
{
    printf("Hola SO!\n");
    ➡ fork();
    printf("Nos vemos!\n");
    return 0;
}
```

- El proceso A se denomina Padre y el proceso B Hijo. Cada uno tiene su propio PID.
- A partir de este punto sus ejecuciones son independientes y el orden de ejecución lo decide el scheduler.
- **¡Importante!** Cada proceso corre en espacios de memoria separados.

¡Importantísimo!

NO SE COMPARTE MEMORIA

Creación de procesos utilizando `fork()`

- ¿Cómo podemos hacer que los procesos creados hagan cosas distintas?

Creación de procesos utilizando `fork()`

- ¿Cómo podemos hacer que los procesos creados hagan cosas distintas?
- ¿Cómo sabe cada proceso si es padre o hijo?

Creación de procesos utilizando `fork()`

- ¿Cómo podemos hacer que los procesos creados hagan cosas distintas?
- ¿Cómo sabe cada proceso si es padre o hijo?

El valor que devuelve `fork()` se ve diferente en el padre y en el hijo.

- En el proceso padre, `fork()` devuelve el valor del PID del hijo recién creado, pero en el proceso hijo esa variable queda con valor 0.
- **Ojo:** 0 NO es el PID del hijo.
- Para el padre, esta es la única forma de conseguir el PID del hijo.

De esta forma, podemos asignar distintos segmentos de código tanto al padre como al hijo.

Creación de procesos utilizando `fork()`

Veamos un demo...

Código en C con fork

Crea un nuevo proceso que es un clon del primero.

Proceso padre

```
int main()
{
    pidOrZero: 1145
    ➔ pid_t pidOrZero = fork();
    if (pidOrZero == 0) {
        Subrutina_proceso_hijo();
    } else {
        Subrutina_proceso_padre();
    }
    exit(EXIT_SUCESS);
}
```

Proceso hijo

```
int main()
{
    pidOrZero: 0
    ➔ pid_t pidOrZero = fork();
    if (pidOrZero == 0) {
        Subrutina_proceso_hijo();
    } else {
        Subrutina_proceso_padre();
    }
    exit(EXIT_SUCESS);
}
```

Código en C con fork

Crea un nuevo proceso que es un clon del primero.

Proceso padre

```
int main()
{
    pid_t pidOrZero = fork();
    if (pidOrZero == 0) {
        Subrutina_proceso_hijo();
    } else {
        Subrutina_proceso_padre();
    }
    exit(EXIT_SUCESS);
}
```

pidOrZero: 1145

Proceso hijo

```
int main()
{
    pid_t pidOrZero = fork();
    if (pidOrZero == 0) {
        Subrutina_proceso_hijo();
    } else {
        Subrutina_proceso_padre();
    }
    exit(EXIT_SUCESS);
}
```

pidOrZero: 0

- **¡Importante!** No tenemos control sobre el orden en que se ejecutan los procesos.

Código en C con fork

Crea un nuevo proceso que es un clon del primero.

Proceso padre

```
int main()
{
    pid_t pidOrZero = fork();
    if (pidOrZero == 0) {
        Subrutina_proceso_hijo();
    } else {
        Subrutina_proceso_padre();
    }
    exit(EXIT_SUCESS);
}
```

pidOrZero: 1145

Proceso hijo

```
int main()
{
    pid_t pidOrZero = fork();
    if (pidOrZero == 0) {
        Subrutina_proceso_hijo();
    } else {
        Subrutina_proceso_padre();
    }
    exit(EXIT_SUCESS);
}
```

pidOrZero: 0

- **¡Importante!** No tenemos control sobre el orden en que se ejecutan los procesos.
- Sólo a modo de ilustración, se muestra un ejemplo donde cada proceso está ejecutando su respectiva función al mismo tiempo.

Identificación de procesos

- `pid_t getppid(void)`: Obtener el PID del padre del proceso actual.
- `pid_t getpid(void)`: Conseguir el PID del proceso actual.

Creación de procesos

fork()

- ¿Qué sucede si el proceso padre termina su ejecución antes que el hijo?

Creación de procesos

fork()

- ¿Qué sucede si el proceso padre termina su ejecución antes que el hijo?
- En ese caso, se dice que el proceso hijo queda **huérfano**.

Creación de procesos

fork()

- ¿Qué sucede si el proceso padre termina su ejecución antes que el hijo?
- En ese caso, se dice que el proceso hijo queda **huérfano**.
- Otro proceso se hace cargo de este proceso huérfano y pasa a ser su padre.

Creación de procesos

fork()

- ¿Qué sucede si el proceso padre termina su ejecución antes que el hijo?
- En ese caso, se dice que el proceso hijo queda **huérfano**.
- Otro proceso se hace cargo de este proceso huérfano y pasa a ser su padre.
- Por lo general, el proceso `init` es el encargado.

Creación de procesos

fork()

- ¿Qué sucede si el proceso padre termina su ejecución antes que el hijo?
- En ese caso, se dice que el proceso hijo queda **huérfano**.
- Otro proceso se hace cargo de este proceso huérfano y pasa a ser su padre.
- Por lo general, el proceso `init` es el encargado.
- Sin embargo, en Linux existen los procesos “**subreaper**”, que son procesos que se pueden autodeclarar como padres de procesos huérfanos que sean descendientes suyos.

Creación de procesos

fork()

Veamos un ejemplo implementado...

Inspección de Procesos Huérfanos en Terminal

Una vez ejecutado `./crear_huerfano`, podemos verificar cómo el kernel reasigna el PPID desde otra terminal:

Inspección de Procesos Huérfanos en Terminal

Una vez ejecutado `./crear_huerfano`, podemos verificar cómo el kernel reasigna el PPID desde otra terminal:

- **Ver PID, nuevo PPID y estado:**

```
ps -eo pid,ppid,stat,cmd | grep crear_huerfano
```

Inspección de Procesos Huérfanos en Terminal

Una vez ejecutado `./crear_huerfano`, podemos verificar cómo el kernel reasigna el PPID desde otra terminal:

- **Ver PID, nuevo PPID y estado:**

```
ps -eo pid,ppid,stat,cmd | grep crear_huerfano
```

- **Ver la jerarquía en árbol:**

```
pstree -p -s <PID_DEL_HIJO>
```

Inspección de Procesos Huérfanos en Terminal

Una vez ejecutado `./crear_huerfano`, podemos verificar cómo el kernel reasigna el PPID desde otra terminal:

- **Ver PID, nuevo PPID y estado:**

```
ps -eo pid,ppid,stat,cmd | grep crear_huerfano
```

- **Ver la jerarquía en árbol:**

```
pstree -p -s <PID_DEL_HIJO>
```

- **Consultar directamente al kernel (/proc):**

```
cat /proc/<PID>/status | grep -iE 'Name|Pid|PPid'
```


Inspección de Procesos Huérfanos en Terminal

Una vez ejecutado `./crear_huerfano`, podemos verificar cómo el kernel reasigna el PPID desde otra terminal:

- **Ver PID, nuevo PPID y estado:**

```
ps -eo pid,ppid,stat,cmd | grep crear_huerfano
```

- **Ver la jerarquía en árbol:**

```
pstree -p -s <PID_DEL_HIJO>
```

- **Consultar directamente al kernel (/proc):**

```
cat /proc/<PID>/status | grep -iE 'Name|Pid|PPid'
```

Limpieza

Para terminar el proceso dormido: `killall crear_huerfano`

Aclaraciones de Fork - Clone

- Cuando se realiza el llamado a `fork()`, por debajo se está llamando a la *syscall* `clone`.

Aclaraciones de Fork - Clone

- Cuando se realiza el llamado a `fork()`, por debajo se está llamando a la *syscall* `clone`.
- Es un mecanismo para realizar la creación de procesos. Podemos determinar sobre qué contextos de ejecución comparten padre e hijo.

Aclaraciones de Fork - Clone

- Cuando se realiza el llamado a `fork()`, por debajo se está llamando a la *syscall* `clone`.
- Es un mecanismo para realizar la creación de procesos. Podemos determinar sobre qué contextos de ejecución comparten padre e hijo.
- Por ejemplo, se puede controlar si se quiere que compartan el espacio virtual, el *stack*, dónde arranca la ejecución, entre otras cosas.

Aclaraciones de Fork - Clone

- Cuando se realiza el llamado a `fork()`, por debajo se está llamando a la *syscall* `clone`.
- Es un mecanismo para realizar la creación de procesos. Podemos determinar sobre qué contextos de ejecución comparten padre e hijo.
- Por ejemplo, se puede controlar si se quiere que compartan el espacio virtual, el *stack*, dónde arranca la ejecución, entre otras cosas.
- Tanto procesos como *threads* utilizan esta *syscall* con sus parámetros correspondientes.

Aclaraciones de Fork - Clone

- Cuando se realiza el llamado a `fork()`, por debajo se está llamando a la *syscall* `clone`.
- Es un mecanismo para realizar la creación de procesos. Podemos determinar sobre qué contextos de ejecución comparten padre e hijo.
- Por ejemplo, se puede controlar si se quiere que compartan el espacio virtual, el *stack*, dónde arranca la ejecución, entre otras cosas.
- Tanto procesos como *threads* utilizan esta *syscall* con sus parámetros correspondientes.
- Para más detalle: `man clone`.

¡Manos a la obra!

Supongamos que Juan tiene 2 hijos, Julieta y Jorge. A su vez Julieta tiene una hija, Jennifer, que nació antes que su tío Jorge. Se requiere la creación y ejecución procesos que emulen la vida de cada uno.

¡Manos a la obra!

Supongamos que Juan tiene 2 hijos, Julieta y Jorge. A su vez Julieta tiene una hija, Jennifer, que nació antes que su tío Jorge. Se requiere la creación y ejecución procesos que emulen la vida de cada uno.

- ¿Cómo podría hacer para que se lancen los procesos en el momento adecuado y sin problemas?

Veamos...

Creación y control de procesos

Parte II

- `pid_t wait(int *status)`: Bloquea al padre hasta que el hijo cambie de estado (si no se indica ningún status). El cambio de estado más común es cuando el hijo termina su ejecución.
- `pid_t waitpid(pid_t pid, int *status, int options)`: Igual a `wait` pero espera al proceso correspondiente al `pid` indicado.
- `void exit(int status)`: Finaliza el proceso actual.

¡Manos a la obra!

Podríamos usar wait.

```
int status;  
// Si termino con errores  
if(wait(&status) < 0){perror("wait");exit(-1);}
```

Aclaraciones de wait

- La `syscall wait()` permite liberar los recursos asociados al hijo.
- Si no se hace esta operación, cuando el proceso hijo muere, continúa en un estado `zombie`.

Aclaraciones de wait

- La `syscall wait()` permite liberar los recursos asociados al hijo.
- Si no se hace esta operación, cuando el proceso hijo muere, continúa en un estado `zombie`.
- Esto significa que la entrada del proceso en la tabla de procesos permanece a pesar de haber terminado su ejecución.
- Sin embargo, si el proceso padre termina, esta operación se hace automáticamente.

La llamada al sistema pause()

Suspende la ejecución del proceso hasta que se reciba **cualquier señal** que ejecute un manejador o termine el proceso.

```
#include <unistd.h>  
int pause(void); // Siempre retorna -1
```

La llamada al sistema pause()

Suspende la ejecución del proceso hasta que se reciba **cualquier señal** que ejecute un manejador o termine el proceso.

```
#include <unistd.h>
int pause(void); // Siempre retorna -1
```

Diferencias clave: pause() vs. wait()

■ Evento de espera:

- pause(): Espera la llegada de **cualquier señal** (ej. SIGINT, SIGALRM).
- wait(): Espera la **terminación/cambio de estado** de un hijo.

La llamada al sistema pause()

Suspende la ejecución del proceso hasta que se reciba **cualquier señal** que ejecute un manejador o termine el proceso.

```
#include <unistd.h>
int pause(void); // Siempre retorna -1
```

Diferencias clave: pause() vs. wait()

- **Evento de espera:**
 - pause(): Espera la llegada de **cualquier señal** (ej. SIGINT, SIGALRM).
 - wait(): Espera la **terminación/cambio de estado** de un hijo.
- **Propósito principal:** Sincronización mediante eventos/señales sin consumir CPU.

La llamada al sistema pause()

Suspende la ejecución del proceso hasta que se reciba **cualquier señal** que ejecute un manejador o termine el proceso.

```
#include <unistd.h>
int pause(void); // Siempre retorna -1
```

Diferencias clave: pause() vs. wait()

- **Evento de espera:**
 - pause(): Espera la llegada de **cualquier señal** (ej. SIGINT, SIGALRM).
 - wait(): Espera la **terminación/cambio de estado** de un hijo.
- **Propósito principal:** Sincronización mediante eventos/señales sin consumir CPU.

¡Atención con las condiciones de carrera!

Si la señal llega justo **antes** de invocar a `pause()`, la señal se procesa y `pause()` bloqueará el proceso indefinidamente.

¡Atención con las condiciones de carrera!

Si la señal llega justo **antes** de invocar a `pause()`, la señal se procesa y `pause()` bloqueará el proceso indefinidamente.

Intuición: La "notificación perdida"

Es el equivalente a esperar un llamado telefónico:

- Si la llamada entra **un segundo antes** de que te sientes a esperar, la llamada se atiende/pierde en ese instante.
- Luego ejecutas `pause()` y te quedas esperando un evento que **ya ocurrió** y no se repetirá.

Volviendo al tema de la memoria

Dijimos que los procesos no comparten memoria.
Veamos el siguiente ejemplo en código...

- ¿Cómo puede suceder que el padre y el hijo guarden diferente información en la misma dirección?

- ¿Cómo puede suceder que el padre y el hijo guarden diferente información en la misma dirección?
- El hijo es una copia de la memoria del padre, así que los punteros referencian la misma dirección virtual...

Volviendo al tema de la memoria

- ¿Cómo puede suceder que el padre y el hijo guarden diferente información en la misma dirección?
- El hijo es una copia de la memoria del padre, así que los punteros referencian la misma dirección virtual...
- ¡Pero no comparten memoria! Las direcciones virtuales de padre e hijo están mapeadas a distinta memoria física.

Volviendo al tema de la memoria

- ¿Cómo puede suceder que el padre y el hijo guarden diferente información en la misma dirección?
- El hijo es una copia de la memoria del padre, así que los punteros referencian la misma dirección virtual...
- ¡Pero no comparten memoria! Las direcciones virtuales de padre e hijo están mapeadas a distinta memoria física.
- ¿No es muy caro hacer copias de toda la memoria cuando se forkea?

- El sistema operativo sólo hace copias *lazy*.

- El sistema operativo sólo hace copias *lazy*.
- Ambos compartirán las mismas páginas físicas hasta que alguna de ellas cambia el contenido.

- El sistema operativo sólo hace copias *lazy*.
- Ambos compartirán las mismas páginas físicas hasta que alguna de ellas cambia el contenido.
- En ese momento se asigna una página física distinta para el proceso que modifica la memoria.

- El sistema operativo sólo hace copias *lazy*.
- Ambos compartirán las mismas páginas físicas hasta que alguna de ellas cambia el contenido.
- En ese momento se asigna una página física distinta para el proceso que modifica la memoria.
- Es decir, sólo se comparten páginas en modo lectura.

Copy on write

- El sistema operativo sólo hace copias *lazy*.
- Ambos compartirán las mismas páginas físicas hasta que alguna de ellas cambia el contenido.
- En ese momento se asigna una página física distinta para el proceso que modifica la memoria.
- Es decir, sólo se comparten páginas en modo lectura.
- Este mecanismo se llama **copy on write**.

Control de procesos

Familia `exec`

La familia de `syscalls` `exec` reemplazan la imagen del proceso actual con una nueva.

Una de las más utilizadas es `execve`.

- `int execve(const char *filename, char *const argv[], char *const envp[]):`
Sustituye la imagen de memoria del programa por la del programa ubicado en `filename`.

Las funciones son: `execl`, `execlp`, `execle`, `execv`, `execvp`, `execve`, `execvpe`.

Cada letra luego del prefijo `exec`, nos indica un significado particular de lo que hace cada función:

- 1: Indica que la función es variádica (aridad indefinida). Toma una secuencia de argumentos que se le pasa a la imagen a reemplazar.

```
$ execl [pathname] [arg1] [arg2] ... [argN] [NULL]
```

Las funciones son: `execl`, `execlp`, `execle`, `execv`, `execvp`, `execve`, `execvpe`.

Cada letra luego del prefijo `exec`, nos indica un significado particular de lo que hace cada función:

- **l**: Indica que la función es variádica (aridad indefinida). Toma una secuencia de argumentos que se le pasa a la imagen a reemplazar.
`$ execl [pathname] [arg1] [arg2] ... [argN] [NULL]`
- **v**: Indica que la función toma un array de punteros a `char` como los parámetros a usar.
`$ execv [pathname] [array args]`

Las funciones son: `execl`, `execlp`, `execle`, `execv`, `execvp`, `execve`, `execvpe`.

Cada letra luego del prefijo `exec`, nos indica un significado particular de lo que hace cada función:

- `e`: Indica que se le pueden pasar variables de entorno, tanto de forma variádica como usando un `array`.

```
$ execve [pathname] [array args] [array env_var]
```


Las funciones son: `execl`, `execlp`, `execle`, `execv`, `execvp`, `execve`, `execvpe`.

Cada letra luego del prefijo `exec`, nos indica un significado particular de lo que hace cada función:

- **e**: Indica que se le pueden pasar variables de entorno, tanto de forma variádica como usando un `array`.

```
$ execve [pathname] [array args] [array env_var]
```

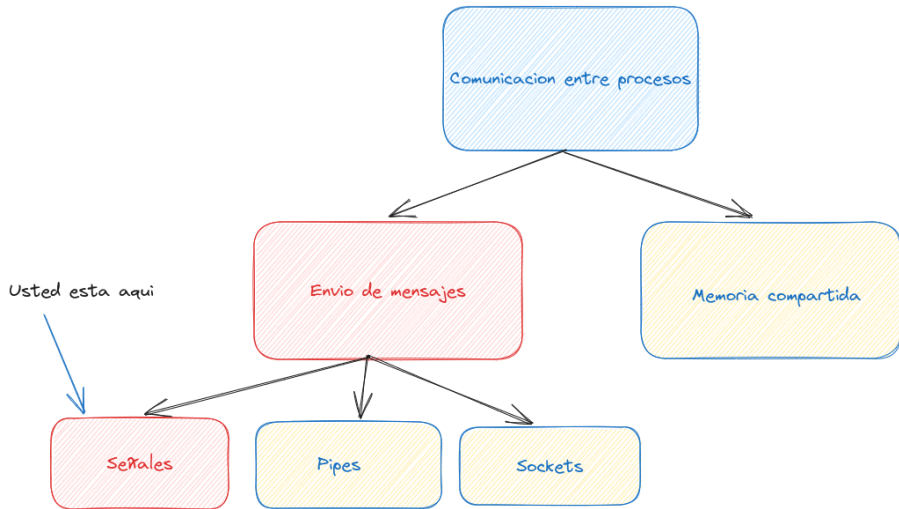
- **p**: Indica que el nombre pasado en `filename`, por defecto lo busque en el `pathname` que indica la variable de entorno `PATH`.

Por ejemplo, si utilizamos `execvp('ls', ['-a'])`, buscará el nombre del comando `ls` según el contenido de la variable `PATH`.

```
$ execvp [filename] [array args]
```

Momento para preguntas

Comunicación entre procesos



- Las **señales** son un mecanismo que incorporan los sistemas operativos basados en POSIX, que permiten notificar a un proceso la ocurrencia de un evento.

- Las **señales** son un mecanismo que incorporan los sistemas operativos basados en POSIX, que permiten notificar a un proceso la ocurrencia de un evento.
- Para implementarlos en C, se utiliza `signal.h`.

- Las **señales** son un mecanismo que incorporan los sistemas operativos basados en POSIX, que permiten notificar a un proceso la ocurrencia de un evento.
- Para implementarlos en C, se utiliza `signal.h`.
- Cada señal es un número, pero comúnmente se les identifica mediante macros.

- Las **señales** son un mecanismo que incorporan los sistemas operativos basados en POSIX, que permiten notificar a un proceso la ocurrencia de un evento.
- Para implementarlos en C, se utiliza `signal.h`.
- Cada señal es un número, pero comúnmente se les identifica mediante macros.
- Ejemplo: SIGINT (señal 2), SIGKILL (señal 9), SIGSEGV (señal 11).

- Las **señales** son un mecanismo que incorporan los sistemas operativos basados en POSIX, que permiten notificar a un proceso la ocurrencia de un evento.
- Para implementarlos en C, se utiliza `signal.h`.
- Cada señal es un número, pero comúnmente se les identifica mediante macros.
- Ejemplo: `SIGINT` (señal 2), `SIGKILL` (señal 9), `SIGSEGV` (señal 11).
- Un usuario puede enviar desde la terminal una señal a un proceso con el comando `kill`. Un proceso puede enviar una señal a otro mediante la `syscall kill()`.

- Las **señales** son un mecanismo que incorporan los sistemas operativos basados en POSIX, que permiten notificar a un proceso la ocurrencia de un evento.
- Para implementarlos en C, se utiliza `signal.h`.
- Cada señal es un número, pero comúnmente se les identifica mediante macros.
- Ejemplo: SIGINT (señal 2), SIGKILL (señal 9), SIGSEGV (señal 11).
- Un usuario puede enviar desde la terminal una señal a un proceso con el comando `kill`. Un proceso puede enviar una señal a otro mediante la `syscall` `kill()`.
- Veamos `man kill`

- Las **señales** son un mecanismo que incorporan los sistemas operativos basados en POSIX, que permiten notificar a un proceso la ocurrencia de un evento.
- Para implementarlos en C, se utiliza `signal.h`.
- Cada señal es un número, pero comúnmente se les identifica mediante macros.
- Ejemplo: SIGINT (señal 2), SIGKILL (señal 9), SIGSEGV (señal 11).
- Un usuario puede enviar desde la terminal una señal a un proceso con el comando `kill`. Un proceso puede enviar una señal a otro mediante la `syscall kill()`.
- Veamos `man kill`
- `kill -L`

- Es posible redefinir el comportamiento de algunas señales usando funciones `void` sin parámetros, comúnmente llamadas `handlers`.

- Es posible redefinir el comportamiento de algunas señales usando funciones `void` sin parámetros, comúnmente llamadas `handlers`.
- Para esto se utiliza la función en C `signal()`, que a su vez usa una `syscall` con el mismo nombre.

- Es posible redefinir el comportamiento de algunas señales usando funciones `void` sin parámetros, comúnmente llamadas `handlers`.
- Para esto se utiliza la función en C `signal()`, que a su vez usa una `syscall` con el mismo nombre.
- Como primer parámetro, indicamos la señal a la que le queremos cambiar su comportamiento.

- Es posible redefinir el comportamiento de algunas señales usando funciones `void` sin parámetros, comúnmente llamadas `handlers`.
- Para esto se utiliza la función en C `signal()`, que a su vez usa una `syscall` con el mismo nombre.
- Como primer parámetro, indicamos la señal a la que le queremos cambiar su comportamiento.
- Como segundo parámetro, le pasamos el nombre de una función, el `handler`.

- Es posible redefinir el comportamiento de algunas señales usando funciones `void` sin parámetros, comúnmente llamadas `handlers`.
- Para esto se utiliza la función en C `signal()`, que a su vez usa una `syscall` con el mismo nombre.
- Como primer parámetro, indicamos la señal a la que le queremos cambiar su comportamiento.
- Como segundo parámetro, le pasamos el nombre de una función, el `handler`.
- **¡Importante!** Algunas señales no se pueden *handlear*, como `SIGKILL`.

- Es posible redefinir el comportamiento de algunas señales usando funciones `void` sin parámetros, comúnmente llamadas `handlers`.
- Para esto se utiliza la función en C `signal()`, que a su vez usa una `syscall` con el mismo nombre.
- Como primer parámetro, indicamos la señal a la que le queremos cambiar su comportamiento.
- Como segundo parámetro, le pasamos el nombre de una función, el `handler`.
- **¡Importante!** Algunas señales no se pueden *handlear*, como `SIGKILL`.

Veamos un ejemplo de código. `signal.c`

¿Qué es una función Async-Signal-Safe?

Es una función que POSIX garantiza que se puede invocar de forma segura **dentro de un manejador de señales** sin causar corrupción de memoria o interbloqueos ([deadlocks](#)).

¿Qué es una función Async-Signal-Safe?

Es una función que POSIX garantiza que se puede invocar de forma segura **dentro de un manejador de señales** sin causar corrupción de memoria o interbloqueos (**deadlocks**).

El peligro: Reentrancia y Locks

Funciones como `printf()` o `malloc()` usan **buffers y locks internos**. Si una señal interrumpe a `printf()` a mitad de ejecución y el manejador vuelve a llamar a `printf()`, el proceso se colgará esperando su propio lock.

¿Qué es una función Async-Signal-Safe?

Es una función que POSIX garantiza que se puede invocar de forma segura **dentro de un manejador de señales** sin causar corrupción de memoria o interbloqueos (**deadlocks**).

El peligro: Reentrancia y Locks

Funciones como `printf()` o `malloc()` usan **buffers y locks internos**. Si una señal interrumpe a `printf()` a mitad de ejecución y el manejador vuelve a llamar a `printf()`, el proceso se colgará esperando su propio lock.

Ejemplos clave (POSIX)

- **Seguras:** `write()`, `read()`, `_exit()`, `sigaction()`, `pause()`.
- **NO seguras:** `printf()`, `malloc()`, `free()`, `exit()`, `sprintf()`.

- Como dijimos previamente, `wait()` espera a que un hijo cambie de estado.

- Como dijimos previamente, `wait()` espera a que un hijo cambie de estado.
- Pero, ¿cómo funciona este mecanismo de espera?

- Como dijimos previamente, `wait()` espera a que un hijo cambie de estado.
- Pero, ¿cómo funciona este mecanismo de espera?
- Cuando un proceso hijo termina su ejecución, envía la señal `SIGCHLD` y el padre la recibe.

- Como dijimos previamente, `wait()` espera a que un hijo cambie de estado.
- Pero, ¿cómo funciona este mecanismo de espera?
- Cuando un proceso hijo termina su ejecución, envía la señal `SIGCHLD` y el padre la recibe.
- No solamente se envía en este caso, sino también cuando ocurre un cambio de estado: que un hijo terminó, que el hijo fue frenado por una señal, o el hijo fue reanudado por una señal.

Capabilities

- En Linux se distinguen procesos con permisos privilegiados (root) y no privilegiados.

Capabilities

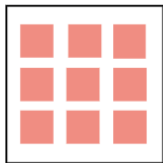
- En Linux se distinguen procesos con permisos privilegiados (root) y no privilegiados.
- Un proceso no root no puede enviar señales a procesos root

Capabilities

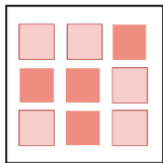
- En Linux se distinguen procesos con permisos privilegiados (root) y no privilegiados.
- Un proceso no root no puede enviar señales a procesos root
- Sin embargo, en Linux es posible permitir este envío, ya que divide los privilegios tradicionalmente asociados con root en distintas unidades llamadas **capabilities**, que pueden ser habilitadas o deshabilitadas.

Capabilities

- En Linux se distinguen procesos con permisos privilegiados (root) y no privilegiados.
- Un proceso no root no puede enviar señales a procesos root
- Sin embargo, en Linux es posible permitir este envío, ya que divide los privilegios tradicionalmente asociados con root en distintas unidades llamadas **capabilities**, que pueden ser habilitadas o deshabilitadas.
- Se puede usar `setcap` para cambiar las **capabilities**, CAP_KILL se denomina a la capacidad para enviar señales a cualquier proceso,



full capabilities



specific capabilities

Concepto: El "Filtro" del Kernel

La **máscara de señales** es un mapa de bits (**bitmask**) almacenado dentro de la estructura del proceso (`task_struct`) en el kernel.

Concepto: El "Filtro" del Kernel

La **máscara de señales** es un mapa de bits (**bitmask**) almacenado dentro de la estructura del proceso (`task_struct`) en el kernel.

- **Bit en 0 (Desbloqueada):** La señal se entrega de inmediato al proceso.
- **Bit en 1 (Bloqueada / Enmascarada):** La señal no se pierde; queda retenida como **pendiente** hasta que se vuelva a desbloquear.

Concepto: El "Filtro" del Kernel

La **máscara de señales** es un mapa de bits (**bitmask**) almacenado dentro de la estructura del proceso (`task_struct`) en el kernel.

- **Bit en 0 (Desbloqueada):** La señal se entrega de inmediato al proceso.
- **Bit en 1 (Bloqueada / Enmascarada):** La señal no se pierde; queda retenida como **pendiente** hasta que se vuelva a desbloquear.

Representación en Bits

SIGKILL (9)	SIGALRM (14)	SIGTERM (15)	SIGINT (2)
0	0	0	1 (<i>Bloqueada</i>)

La llamada al sistema `sigsuspend`

```
int sigsuspend(const sigset_t *mask);
```

La llamada al sistema sigsuspend

```
int sigsuspend(const sigset_t *mask);
```

Comportamiento Atómico

Realiza las siguientes acciones en un **único paso indivisible** dentro del kernel:

1. Reemplaza temporalmente la máscara del proceso por `mask` (donde la señal esperada **no** debe estar bloqueada).
2. Pone a dormir el proceso a la espera de una señal que ejecute un manejador o lo termine.

API - S.O.

Sincronización Segura (sigsuspend)

La llamada al sistema sigsuspend

```
int sigsuspend(const sigset_t *mask);
```

Comportamiento Atómico

Realiza las siguientes acciones en un **único paso indivisible** dentro del kernel:

1. Reemplaza temporalmente la máscara del proceso por `mask` (donde la señal esperada **no** debe estar bloqueada).
2. Pone a dormir el proceso a la espera de una señal que ejecute un manejador o lo termine.

Retorno y Restauración

- Cuando el manejador de la señal termina, `sigsuspend()` retorna `-1` y asigna `errno = EINTR`.
- Al retornar, el kernel **restaura automáticamente** la máscara de

Funciones para Modificar el Tipo `sigset_t`

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!
- `sigfillset(&set)`: Incluye todas las señales en el conjunto (bits en 1).

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!
- `sigfillset(&set)`: Incluye todas las señales en el conjunto (bits en 1).
- `sigaddset(&set, SIGINT)`: Añade una señal específica al conjunto.

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!
- `sigfillset(&set)`: Incluye todas las señales en el conjunto (bits en 1).
- `sigaddset(&set, SIGINT)`: Añade una señal específica al conjunto.
- `sigdelset(&set, SIGINT)`: Remueve una señal específica del conjunto.

Funciones para Modificar el Tipo sigset_t

Antes de cambiar la máscara del proceso, debemos construir el conjunto de señales en memoria:

- `sigemptyset(&set)`: Vacía el conjunto (pone todos los bits en 0).
¡Siempre inicializar con esto!
- `sigfillset(&set)`: Incluye todas las señales en el conjunto (bits en 1).
- `sigaddset(&set, SIGINT)`: Añade una señal específica al conjunto.
- `sigdelset(&set, SIGINT)`: Remueve una señal específica del conjunto.
- `sigismember(&set, SIGINT)`: Verifica si la señal está en el conjunto (retorna 1 o 0).

API - S.O.

Aplicar la Máscara al Proceso (sigprocmask)

La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```


La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Modos de operación (how)

- SIG_BLOCK: Bloquea las señales pasadas en set (las añade a la máscara actual).

La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Modos de operación (how)

- SIG_BLOCK: Bloquea las señales pasadas en set (las añade a la máscara actual).
- SIG_UNBLOCK: Desbloquea las señales pasadas en set (las quita de la máscara actual).

La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Modos de operación (how)

- SIG_BLOCK: Bloquea las señales pasadas en set (las añade a la máscara actual).
- SIG_UNBLOCK: Desbloquea las señales pasadas en set (las quita de la máscara actual).
- SIG_SETMASK: Reemplaza la máscara actual por completo con set.

API - S.O.

Aplicar la Máscara al Proceso (sigprocmask)

La llamada al sistema sigprocmask

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

Modos de operación (how)

- SIG_BLOCK: Bloquea las señales pasadas en set (las añade a la máscara actual).
- SIG_UNBLOCK: Desbloquea las señales pasadas en set (las quita de la máscara actual).
- SIG_SETMASK: Reemplaza la máscara actual por completo con set.

Parámetro oldset

Si no es NULL, el kernel guarda allí la máscara que estaba activa antes del cambio (útil para restaurarla después).

Usando strace

strace es una herramienta que nos permite generar una traza legible de las llamadas al sistema usadas por un programa dado.

Ejemplo de strace

```
$ strace -q echo hola > /dev/null
```

Algunas opciones útiles:

- **-q**: Omite algunos mensajes innecesarios.
- **-o <archivo>**: Redirige la salida a <archivo>.
- **-f**: Muestra también la traza de los procesos hijos.

Usando strace

strace es una herramienta que nos permite generar una traza legible de las llamadas al sistema usadas por un programa dado.

Ejemplo de strace

```
$ strace -q echo hola > /dev/null
execve("/bin/echo", ["echo", "hola"], [/* 70 vars */]) = 0
write(1, "hola\n", 5)                = 5
exit_group(0)                        = ?
```

- `execve()` convierte el proceso en una instancia nueva de `./bin/echo` y devuelve 0 indicando que no hubo error.
- `write()` escribe en pantalla el mensaje y devuelve la cantidad de caracteres escritos (5).
- `exit_group()` termina la ejecución(y de todos sus *threads*, de haberlos) y no devuelve ningún valor.

strace y hello en C

Probemos `strace` con nuestra versión en C del programa.

hello.c

```
#include <unistd.h>

int main(int argc, char* argv[]) {
    write(1, "Hola S0!\n", 9);
    return 0;
}
```

Vamos a compilar estáticamente:

Compilación de hello.c

```
gcc -static -o hello hello.c
```

strace y hello en C

strace de hello.c

```
$ strace -q ./hello
execve("./hello", [ "./hello" ], [ /* 17 vars */ ]) = 0
uname({sys="Linux", node="nombrehost", ...}) = 0
brk(0)                                     = 0x831f000
brk(0x831fcb0)                             = 0x831fcb0
set_thread_area({entry_number:-1 -> 6, base_addr:0x831f830
...}) = 0
brk(0x8340cb0)                             = 0x8340cb0
brk(0x8341000)                             = 0x8341000
write(1, "Hola SO!\n", 9)                  = 9
exit_group(0)                              = ?
```

¿Qué es todo esto?

Filtrado y Seguimiento de Procesos

- `-f`: Sigue a los procesos hijos generados por `fork()` o `clone()`.

Filtrado y Seguimiento de Procesos

- `-f`: Sigue a los procesos hijos generados por `fork()` o `clone()`.
- `-p <PID>`: Se engancha ([attach](#)) a un proceso que ya está corriendo.

Filtrado y Seguimiento de Procesos

- `-f`: Sigue a los procesos hijos generados por `fork()` o `clone()`.
- `-p <PID>`: Se engancha ([attach](#)) a un proceso que ya está corriendo.
- `-e trace=<grupo|syscalls>`: Filtra por llamadas o grupos específicos:

```
$ strace -e trace=process,signal ./programa
```

```
$ strace -e trace=open,read,write ./programa
```

Filtrado y Seguimiento de Procesos

- `-f`: Sigue a los procesos hijos generados por `fork()` o `clone()`.
- `-p <PID>`: Se engancha ([attach](#)) a un proceso que ya está corriendo.
- `-e trace=<grupo|syscalls>`: Filtra por llamadas o grupos específicos:

```
$ strace -e trace=process,signal ./programa
```

```
$ strace -e trace=open,read,write ./programa
```

Diagnóstico y Métricas

- `-c`: Profiling de syscalls (tiempo consumido, llamadas totales y errores).

Filtrado y Seguimiento de Procesos

- `-f`: Sigue a los procesos hijos generados por `fork()` o `clone()`.
- `-p <PID>`: Se engancha ([attach](#)) a un proceso que ya está corriendo.
- `-e trace=<grupo|syscalls>`: Filtra por llamadas o grupos específicos:

```
$ strace -e trace=process,signal ./programa
```

```
$ strace -e trace=open,read,write ./programa
```

Diagnóstico y Métricas

- `-c`: Profiling de syscalls (tiempo consumido, llamadas totales y errores).
- `-T`: Muestra el tiempo que tardó el kernel en resolver cada llamada.

Filtrado y Seguimiento de Procesos

- `-f`: Sigue a los procesos hijos generados por `fork()` o `clone()`.
- `-p <PID>`: Se engancha ([attach](#)) a un proceso que ya está corriendo.
- `-e trace=<grupo|syscalls>`: Filtra por llamadas o grupos específicos:

```
$ strace -e trace=process,signal ./programa
```

```
$ strace -e trace=open,read,write ./programa
```

Diagnóstico y Métricas

- `-c`: Profiling de syscalls (tiempo consumido, llamadas totales y errores).
- `-T`: Muestra el tiempo que tardó el kernel en resolver cada llamada.
- `-o salida.txt`: Redirige toda la traza a un archivo de texto.

Momento para preguntas

Hoy vimos...

- ¿Cómo interactuamos con el SO? → Syscalls y wrapper functions.
 - Creación de procesos: `fork()`
 - Identificación de procesos: `getpid()` y `getppid()`
 - Control de procesos: `wait()`, `waitpid()`, `exit()` y la familia `exec`.
- Uso de señales con `signal()` y `kill()`.
- Ingeniería inversa con `strace`.

Hoy vimos...

- ¿Cómo interactuamos con el SO? → Syscalls y wrapper functions.
 - Creación de procesos: `fork()`
 - Identificación de procesos: `getpid()` y `getppid()`
 - Control de procesos: `wait()`, `waitpid()`, `exit()` y la familia `exec`.
- Uso de señales con `signal()` y `kill()`.
- Ingeniería inversa con `strace`.

Cómo seguimos...

- Pueden hacer toda la primera parte de la guía 1.
- Pueden comenzar el taller de [syscalls](#).
- Siguiendo clases → Comunicación Inter-procesos ó IPC.